Prompt de instrução — migrar a automação de WhatsApp (Z-API) para Telegram e fazer a proposta entrar sozinha no gc-sistema

> Documento de instrução para abrir uma sessão nova de trabalho. Levantado em 16/09/2026 contra o estado real do n8n Cloud e dos dois repositórios — não é proposta, é o ponto de partida verificado.
>
> **Revisado em 16/09/2026, depois do fechamento da sprint 4.** A revisão mudou o objetivo: não é só trocar o canal, é fechar o caminho **proposta chega no Telegram → OCR lê o documento inteiro, itens inclusive → a proposta aparece cadastrada no sistema**. Três decisões foram tomadas (Seção 4) e um trecho do levantamento original ficou velho — está corrigido em 0.4.
>
> **Segunda revisão em 21/09/2026**, conferindo o documento linha a linha contra o DDL do gc-sistema. Achou um erro que derrubava a Fase 5 (`itens.valor_total` e `itens.area_m2` são **colunas geradas** — 0.8) e mais três restrições de `itens` que ninguém tinha registrado. As seis decisões em aberto da Seção 9 foram fechadas, viraram a Seção 4 e mais quatro nasceram da própria revisão. O que mudou está marcado com **[rev. 21/09]** ao longo do texto.

Como usar

Cole a partir da Seção 1 numa sessão nova. A Seção 0 é contexto já verificado: **não re-descobrir, não re-auditar**. Se algo dela não bater com o que você encontrar, pare e avise — significa que o estado mudou desde 16/09/2026.

Repositórios envolvidos:

- **`/Users/a1234/gc-sistema`** — o sistema. Next.js 14 (App Router) + Supabase, branch `feature-dev-breno-automacao`. **Leia o `CLAUDE.md` dele antes de escrever uma linha** — a regra do `npm run validar` e as três obrigações que vêm com ela valem integralmente.
- **`/Users/a1234/Documents/antigravity/keen-mendel`** — repositório de documentação, espelhado em `~/Library/Mobile Documents/com~apple~CloudDocs/keen-mendel/`. Toda edição de `.md` / `.pdf` vai nas duas cópias.
- **n8n Cloud** — `https://ederbox.app.n8n.cloud`, operado pelas ferramentas `n8n-mcp`.

[rev. 21/09] Caminhos `keen-mendel/relatorios/…` são do keen-mendel, não do gc-sistema. Não existe `keen-mendel/relatorios/` no gc-sistema. Onde este documento escreve `keen-mendel/relatorios/...`, o caminho completo é `/Users/a1234/Documents/antigravity/keen-mendel/relatorios/...`. Os oito arquivos citados na Seção 10 foram conferidos e existem.

[rev. 21/09] Antes de abrir a sessão: o `n8n-mcp` não enxerga o gc-sistema. O servidor está configurado **só no escopo do keen-mendel** — uma sessão aberta em `/Users/a1234/gc-sistema` não tem nenhuma ferramenta n8n, e as Fases 0, 2, 3, 4, 5 e os pedaços de n8n da 6 e da 8 ficam inexecutáveis. **Decisão 13 da Seção 4: acrescentar o `n8n-mcp` ao `.mcp.json` do gc-sistema** e trabalhar tudo de uma sessão só, já que o documento manda ler o `CLAUDE.md` daqui. Confira antes de começar: se não houver ferramenta `n8n_*` disponível, **pare e resolva o setup** — não tente contornar editando JSON de workflow na mão.

Nota de nomenclatura: **não existe pasta `gc-dev`**. `gc-dev` é o *projeto Supabase* (`gzbmhgnpoehormnidmgg`) contra o qual o gc-sistema roda. `gc-prod` está pausado e fora de escopo — o `scripts/gc-dev-guard.mjs` aborta qualquer script que abra conexão com outro ref.

---

0. Estado verificado em 16/09/2026

0.1 Os 8 workflows e onde o Z-API está

WORKFLOW	ID	NÓS	ATIVO	PONTOS Z-API
Ingestão via WhatsApp (Z-API)	`pmysfzlAQnKeiLWY`	24	sim	2 + webhook
Ingestão e Extração	`lsleLGM6t9iKcSLT`	23	sim	6
Triagem (Contrato/Proposta)	`MLoVZf7ypPS6Zy3H`	23	sim	4
Integração Oficial (Contrato)	`dOt8aiCX2OCr08RH`	15	sim	4
Integração Oficial (Proposta)	`I4wdVriPgS2W5MZP`	11	sim	4
Resolução Manual de Revisão	`RikR8XMy9tw8B8O5`	16	sim	4
Notificação de Erro	`loR10oopB9vpH22K`	5	sim	1
SLA Gatilhos de Execução (5 dias)	`TksH6ervKPA7Liar`	8	**não**	**não auditado**

[rev. 21/09] A conta certa é 25 nós Z-API, e a versão anterior dizia "23 de envio + 1 de download" = 24. A coluna soma `2+6+4+4+4+4+1 = 25`. A diferença é o `Z-API - Confirmar Recebimento`, que fica dentro da Ingestão e some junto com ela na Fase 4. A divisão que importa é por fase:

- **23 nós de envio** espalhados pelos **6 workflows da Fase 3** (`6+4+4+4+4+1`) — são esses que viram chamada ao `Notificar`.
- **2 nós na Ingestão** (`pmysfzlAQnKeiLWY`): 1 de envio (`Confirmar Recebimento`) e 1 de download (`Baixar Arquivo do Z-API`). Não são migrados — o workflow inteiro é substituído por um novo na Fase 4 e fica intacto como rollback até a Fase 8.

O SLA está desativado e não foi aberto — auditá-lo é a primeira tarefa da Fase 3, e se tiver Z-API os números acima sobem.

A migração de canal **não é um workflow, são seis.** Quem olhar só o "Ingestão via WhatsApp" vai subdimensionar o trabalho em 90%.

0.2 Contratos de payload entre os workflows

Não mudar sem necessidade — são a costura que o split de agosto deixou pronta.

```
POST /webhook/obraminds/upload-documento
  { empresa_id, tipo_documento, arquivo_url, obra_id, numero_contrato, telefone }

POST /webhook/obraminds/resposta-revisao
  { empresa_id, obra_id, telefone, resposta_texto }

Execute Sub-workflow → Triagem (inputSource: passthrough)
  { documentoId, empresaId, telefone, obraId, numeroContrato, tipoDocumento,
    valorContrato, itens, propostaReferenciada, vinculoOk, origem, ... }
```

0.3 Onde o telefone vive — os cinco lugares

Esta é a lista que define o tamanho real da migração de canal:

1. Chave de lookup em `contatos_whatsapp.telefone` (2 nós: documento e texto).
2. Campo `telefone` no payload dos dois webhooks e no objeto que vai para a Triagem.
3. Destino de envio (`{ phone: ... }`) nos 23 nós Z-API.
4. **Persistido dentro de `documentos_processamento.dados_extraidos->>telefone`** e usado como *chave de correlação* em `Buscar Documento Pendente` (`...&dados_extraidos->>telefone=eq.{{...}}&order=created_at.desc&limit=1`).
5. Hardcoded como destino de admin — `558598202307`, em ~12 nós, **nunca confirmado como correto**.

O item 4 é o que morde: a correlação resposta↔documento passa por dentro de um campo JSON que hoje carrega um telefone. Trocar o canal sem tratar isso quebra a resolução manual de revisão de forma silenciosa.

0.4 Banco (gc-dev `gzbmhgnpoehormnidmgg`)

- `documentos_processamento` — `20260901120909_documentos_processamento.sql`. CHECK de status: `('PENDENTE','ERRO_VALIDACAO','REVISAO_HUMANA','APROVADO')`.
- `contatos_whatsapp` — `20260901120910_contatos_whatsapp.sql`. `telefone text not null unique`, `empresa_id`, `obra_id`, `numero_contrato`.
- Bucket `documentos-processamento` — `20260901120911`.
- **13 migrations** em `supabase/migrations/`; a última aplicada é `20260908120000_orcamentos_historico.sql`.
- **Corrigido na revisão:** o `supabase migration repair --status applied` das dez versões **já foi executado em gc-dev**, em 2026-09-05, no bloco 4.1 — `migration list` mostra 13/13 alinhadas e `db`

`push` em gc-dev não precisa mais de preparo. A pendência continua valendo **só para gc-prod**, quando despausar. Atenção: o `CLAUDE.md` e o `supabase/README.md` ainda descrevem o repair como pendência aberta, sem dizer que é de prod. Corrigir os dois textos é item da Fase 1 — é um documento contradizendo outro, e quem ler primeiro o `CLAUDE.md` vai travar sem motivo.

0.5 O painel não enxerga a automação

Não existe nenhuma rota em `src/app/(app)/` para `documentos_processamento` nem para `contatos_whatsapp`. As duas tabelas aparecem **só** em `src/lib/supabase/types.ts` (tipo gerado). Hoje a automação escreve direto via REST com a service key e ninguém no sistema vê o que ela está fazendo. É metade do "integrar com o nosso sistema".

0.6 Fontes locais — cuidado

`keen-mendel/relatorios/v0_zapi_whatsapp/obraminds_ingestao_whatsapp_zapi.json` está **desatualizado** (15 nós; o workflow vivo tem 24). Use o n8n como fonte de verdade, não esse arquivo.

Docs que valem a leitura, nesta ordem: `keen-mendel/relatorios/v0.1/fluxo_workflows_v0.1.md` (mapa dos workflows), `keen-mendel/relatorios/md/fluxo_e_campos_extracao.md` (o que a IA extrai hoje, §1.3 e a tabela de campos), `keen-mendel/relatorios/md/fluxo_vinculo_conversacional.md` (§3 e §4 — o bloqueador que o Telegram resolve), `keen-mendel/relatorios/md/modelo_vinculo_contrato_propostas.md` (DDL pendente).

0.7 O que a sprint 4 deixou pronto — e por que isso muda o plano

Fechada em 2026-09-14, oito blocos (4.1 a 4.8), código na `main` desde 2026-09-10 (PR #3). Detalhe em `docs/sprint-4/sprint-4-apresentacao-gestao.md` e `docs/sprint-4/4.1` a `4.8`.

O que interessa para esta rodada:

- **O módulo de Propostas existe inteiro**: `/propostas` (listagem com busca, filtros, período e paginação), `/propostas/nova`, `/propostas/[id]` (seis abas), `/propostas/[id]/editar`, `/api/export/propostas`. Server Actions em `src/app/(app)/propostas/**/actions.ts`.
- **As regras de negócio estão isoladas e testadas** em `src/lib/propostas.ts` (soma dos percentuais, desconto, vencimento, transições de status, `mensagemDeErroProposta`) e `src/lib/historico.ts`. São helpers puros — a rota de ingestão da Fase 6 **reusa esses mesmos helpers**, não reescreve as regras em JavaScript de nó n8n.
- **Status de proposta tem histórico** (`propostas.historico`, jsonb append-only, `20260905180000_propostas_historico.sql`, com CHECK de que é array). Proposta criada pela automação também precisa nascer com rastro — hoje o INSERT do n8n não grava nada disso.
- **A aba "Itens" de `/propostas/[id]` é um placeholder**: "Itens da proposta chegam no Sprint 5" (`src/app/(app)/propostas/[id]/page.tsx:118`). A tela dos itens é da sprint 5, que **corre em paralelo** com esta rodada — ver Seção 4, decisão 3.
- **Baseline do `npm run validar`**, última rodada completa em 2026-09-10 contra gc-dev. É contra estes números que os seus vão ser comparados:

CAMADA	BASELINE
estático	`tsc --noEmit` limpo, lint sem warnings
unitário	**51 casos, 0 falhas**
build	**31 rotas** (`scripts/rotas-esperadas.txt`)
runtime	**41/41 rotas** (`scripts/validacao-rotas.json`)
dados	**12/12 checagens** sob RLS
escrita	**38/38 passos**
navegador	**24/24 passos**, console limpo, 13 screenshots

[rev. 21/09] Baseline reconferida contra os arquivos: `scripts/rotas-esperadas.txt` tem 31 linhas e `scripts/validacao-rotas.json` tem 41 entradas (16 delas com `"perfil"`). Os números batem. Atenção a um detalhe de nomenclatura que o `CLAUDE.md` deixa ambíguo: a chave do JSON é **`"path"`** , não `"rota"` , e as asserções ficam em **`"esperaHtml"`** .

- **Auditoria de cobertura** em `docs/tecnicos/auditoria-cobertura-sprint-4.md` : leia antes de mexer nos scripts de validação. Ela documenta o skip silencioso que fez três checagens de permissão pararem de rodar devolvendo exit 0 — o modo exato de falhar que você precisa não repetir ao acrescentar passos novos.

0.8 O que o schema de **`propostas`** e **`itens`** exige de quem grava

Isto é o contrato que a rota de ingestão (Fase 6) tem de respeitar. Levantado do DDL, não da memória.

`propostas` (`20260424121550_initial.sql` §4, alterada por `20260511151924_revisao_schema.sql` e `20260905180000_propostas_historico.sql`):

- `obra_id` **not null**, com FK composta `(obra_id, empresa_id)` → a obra tem de ser da mesma empresa. **Sem obra confirmada não existe proposta** — o documento fica em `REVISAO_HUMANA` , como já acontece hoje.
- `unique (empresa_id, numero)` — número repetido estoura `propostas_empresa_id_numero_key` , já traduzido por `mensagemDeErroProposta` .
- `valor_final` é **coluna gerada** (`valor_total - desconto`): nunca inserir.
- `propostas_desconto_valido` : `desconto <= valor_total` .
- `pct_sinal` , `pct_fd` , `pct_entrega_material` , `pct_medicao_instalacao` são **frações entre 0 e 1**, não percentuais — 50% grava `0.5` . `propostas_pct_soma` exige soma `<= 1.0` . Os helpers `pctToFraction` / `fractionToPct` de `src/lib/propostas.ts` fazem a conversão.
- `status` CHECK `('rascunho','enviada','aprovada','rejeitada')` .
- `propostas_rejeitada_motivo` : `motivo_rejeicao` preenchido **se e só se** status é `rejeitada` , e só com um dos oito valores do CHECK.
- `historico` jsonb not null, sempre array.

`itens` (`20260424121550_initial.sql` §"itens", **muito alterada** por `20260511151924_revisao_schema.sql` §8 — a revisão de 21/09 encontrou aqui o erro que derrubava a Fase 5):

- Já tem **`proposta_id`** e FK composta `(proposta_id, empresa_id, obra_id)` — o vínculo proposta↔item não precisa de migration nova.
- `empresa_id` e `obra_id` são **not null** no item também.

Colunas graváveis — é nelas, e só nelas, que a ingestão escreve: `numero`, `tipo`, `linha`, `acabamento`, `localizacao`, `descricao`, `quantidade`, `unidade`, `valor_unit`, `largura`, `altura`, `vidros`, `observacao`, `foto_url`.

[rev. 21/09] Colunas GERADAS — nunca inserir, e a consequência é maior do que parece:

```
-- 20260511151924_revisao_schema.sql:284-293
valor_total numeric(14,2) generated always as (valor_unit * quantidade) stored
area_m2     numeric(14,4) generated always as (largura * altura * quantidade) stored
```

`itens.valor_total` **não é um campo que se grava, é o produto `valor_unit × quantidade`**. A versão anterior deste documento listava as duas como "campos disponíveis" e a Seção 3.5 mandava, para item sem unitário, "voltar `null` e deixar `valor_total`". **Isso é impossível:** com `valor_unit` nulo o `valor_total` gerado sai nulo junto, e o item entra no banco sem nenhum valor — perdendo exatamente o dado que a Fase 5 existe para capturar. A regra correta está na Seção 3.5, reescrita, e na decisão 10 da Seção 4.

[rev. 21/09] Mais três restrições que ninguém tinha registrado:

RESTRIÇÃO	ONDE	O QUE QUEBRA
`unidade` CHECK `('QTD','M2')`	`revisao_schema.sql:280`	`UN`, `PÇ`, `m`, `ML` **rejeitam o insert**. O `ML` foi dropado de propósito na revisão de schema (`update itens set unidade='QTD' where unidade='ML'`)
`numero` é **`integer`** e tem `unique (proposta_id, numero) where proposta_id is not null`	`initial.sql` + `revisao_schema.sql:298`	`"1.1"` ou `"1A"` não cabem em integer; item numerado repetido no PDF estoura o índice. Vários `numero` nulos convivem (Postgres não deduplica nulo no unique)
`item_vinculo_xor`	`initial.sql`	Item nunca pode ter `proposta_id` **e** `contrato_id` ao mesmo tempo. Não morde nesta rodada; morde na Seção 8, quando o contrato também passar pela rota

Nenhuma delas exige migration nova — o schema comporta a proposta e os itens vindos do OCR, como a versão anterior já dizia. O que elas exigem é **regra explícita** no prompt da Fase 5 e no helper da Fase 6.

0.9 Onde o caminho quebra hoje, para o objetivo desta rodada

Quatro buracos, em ordem de quanto custam:

1. **O workflow 4b não grava itens.** `Obraminds - Integração Oficial (Proposta)` (`I4wdVriPgS2W5MZP`, 11 nós) cria a linha em `propostas`, notifica e marca o documento — e para aí. O 4a, de contrato, cria `contratos` e `itens` e `execucao`. A proposta chega ao sistema como uma casca: número, valor, obra, nada de itens.
2. **A extração lê pouco de cada item.** Segundo `fluxo_e_campos_extracao.md`, o prompt do Gemini devolve por item apenas `descricao`, `quantidade` e `valor_unitario`. O PDF de proposta traz, por item, **`Item`**, **`Tipo`**, **`Linha`**, **`Quantidade`**, **`Acabamento`**, **`Localização`**, **`Valor unit.`**, **`Valor total`** e a descrição livre embaixo — conferido em `keen-mendel/relatorios/documentos-fonte/PROP. COM. - EF _ ADITIVOS_15 12 2025.pdf`, 12 itens em 6 páginas. Tudo isso tem coluna em `itens` (0.8) e hoje se perde.
3. **A gravação não passa pelas regras do sistema.** O n8n insere direto por REST com a service key, que bypassa RLS e não conhece `src/lib/propostas.ts`. Uma proposta criada pela automação pode nascer com soma de percentuais inválida, sem histórico e com status fora do fluxo, e a tela vai mostrar isso como se fosse dado bom.
4. **`PROPOSTA_REGISTRADA` não existe no CHECK.** O nó `Supabase - Atualizar (PROPOSTA_REGISTRADA)` do 4b escreve um status que o CHECK de `documentos_processamento` não aceita (0.4) — reconferido em 21/09 no DDL: os quatro valores aceitos são `PENDENTE`, `ERRO_VALIDACAO`, `REVISAO_HUMANA` e `APROVADO`. **O update falha hoje**; o que não se sabe é se com erro visível ou em silêncio. O que fazer está decidido (Seção 4, decisão 6); o que falta é olhar o 4a e anotar o achado.

---

1. Objetivo

Uma proposta enviada no Telegram tem de aparecer cadastrada no gc-sistema, com os itens, sem ninguém digitar nada. A troca de canal é o meio; o fim é esse.

Cinco entregas:

1. **Camada de canal**: um sub-workflow `Notificar` único, para que os 6 workflows parem de falar o protocolo do canal diretamente.
2. **Telegram como canal ativo**, com o Z-API desligável sem tocar em 23 nós de novo.
3. **Extração completa dos itens** da proposta — os campos da Seção 0.9 item 2.
4. **Rota de ingestão no gc-sistema**: a automação passa a gravar **pelo sistema**, com as regras de `src/lib/propostas.ts` aplicadas e histórico gravado, em vez de INSERT direto.
5. **Painel**: telas de acompanhamento dos documentos em processamento e de cadastro dos contatos autorizados.

Não faz parte desta rodada: o fluxo conversacional de vínculo com botões (Seção 8).

2. Restrições inegociáveis

- **Só gc-dev.** Nada de `gc-prod` — migration, `db push`, seed, types, validação. Se uma tarefa parecer exigir gc-prod, **pare e pergunte**.
- **`npm run validar` completo** antes de declarar qualquer task do gc-sistema pronta, com os números reais registrados no documento de entrega da fase, em `docs/automacao/` (a trilha não usa blocos `m.n` — ver `docs/README.md`), e o que não rodou listado nominalmente como pendência. Compare com a baseline de 0.7.
- **Não commitar nem dar push sem pedido explícito.** O trabalho fica no working tree.
- **Não apagar nada até o teste end-to-end passar** — nós órfãos, workflow antigo, colunas. É o procedimento que funcionou nas três fases do split de agosto.
- **As credenciais estão coladas em texto puro** nos nós (o plano do n8n não tem Variables). Ao editar esses nós, não deixe nenhuma delas vazar para documento, log ou commit. O token do bot Telegram **vai numa Credential do n8n**, não num nó HTTP. O segredo da rota de ingestão (Fase 6) vai em `.env.local` e numa Credential, nunca no corpo de um nó.

3. As quatro mudanças de fundo

Antes do plano, o que muda de verdade. Quem tratar isso como "trocar a URL do endpoint" vai entregar algo quebrado.

3.1 Identidade

WhatsApp entrega um telefone E.164 — conhecido, estável, que a empresa já tem cadastrado. Telegram entrega um `chat_id` numérico opaco. **O bot não recebe o telefone**, a não ser que a pessoa compartilhe o contato explicitamente (teclado `request_contact`).

Consequência: o cadastro de contatos deixa de ser "digite o telefone do cliente" e passa a exigir um onboarding — a pessoa fala com o bot, o bot descobre o `chat_id`, alguém vincula esse `chat_id` a uma empresa/obra. Isso é schema novo **e** tela nova, e é a razão de a Fase 7 não ser opcional.

3.2 Download do arquivo

Z-API entrega `document.documentUrl` pronto para baixar. Telegram entrega `file_id`, que exige `getFile` e depois `https://api.telegram.org/file/bot<token>/<file_path>` — dois passos onde hoje há um.

E há um limite novo: **a Bot API não baixa arquivos acima de 20 MB.** Contratos escaneados passam disso com facilidade. Hoje não existe nenhum tratamento de tamanho no fluxo, porque o Z-API não exigia. Isso é comportamento novo a construir, não uma adaptação.

3.3 Resposta do humano

Hoje a resposta de revisão é texto livre lido por um parser cego: `Montar Dados Resolvidos` trata **qualquer texto** como número de contrato. O Telegram oferece `reply_to_message` e inline keyboard (`callback_query`), que resolvem exatamente o bloqueador descrito em

`fluxo_vinculo_conversacional.md` §3.1–3.2.

Não implemente isso agora. Mas **não feche a porta**: o contrato de entrada do `Notificar` deve prever um campo de ações/opções desde o início, mesmo que a v1 só mande texto.

3.4 Quem grava deixa de ser o n8n

Hoje o n8n é um segundo aplicativo escrevendo na mesma base, com service key, sem passar por nenhuma regra do sistema (0.9 item 3). Depois desta rodada ele vira um **cliente** do gc-sistema: manda o documento extraído para uma rota de ingestão e recebe de volta o id da proposta ou o motivo da recusa.

O que isso compra, e é o motivo da decisão: as regras de proposta passam a existir **num lugar só**. Quando a soma dos percentuais mudar em `src/lib/propostas.ts`, a automação muda junto, sem ninguém lembrar de abrir o n8n. E a ingestão fica coberta pelas camadas de escrita e de dados do `npm run validar`, que é onde o projeto já sabe provar que algo funciona.

O que isso custa: um salto de rede a mais e um segredo a gerenciar. Aceito.

3.5 Os itens são o conteúdo, não um detalhe

Uma proposta sem itens não serve para o que vem depois — é do item que saem o contrato, a execução e a medição. Fazer o OCR ler os itens completos (0.9 item 2) e gravá-los com `proposta_id` é o que transforma "a automação registrou alguma coisa" em "a proposta está no sistema".

Três armadilhas específicas:

- **A soma dos itens já é conferida** pela Triagem (`quantidade × valor_unitario` vs. `valor_total`, tolerância de 1%, `divergeValor`). Não duplique essa regra na rota de ingestão: ela **rejeita** o que chegar divergente, não recalcula por conta própria.
- **[rev. 21/09 — invertida] Item sem `valor_unit`: o schema obriga a inferir.** O PDF de referência traz vários itens só com "Valor total", e a versão anterior deste documento mandava gravar `valor_unit: null` e deixar o `valor_total`. Não dá: `itens.valor_total` é **coluna gerada** a partir de `valor_unit × quantidade` (0.8). Gravar o unitário nulo não é "ser conservador", é **jogar fora o único valor que o documento tem** para aquele item.

  A regra, portanto, inverte-se (decisão 10 da Seção 4): **a ingestão infere `valor_unit = valor_total_lido / quantidade`** e registra a inferência em `itens.observacao`, com texto fixo e reconhecível — algo como `[valor unitário inferido de R$ X / N]`. O `valor_total` lido do documento **não é gravado**; ele serve de conferência contra o gerado, e divergência acima de 1% manda o documento para `REVISAO_HUMANA` em vez de gravar número errado em silêncio.

  Casos que a inferência não cobre e que vão para `REVISAO_HUMANA`: `quantidade` nula ou zero (divisão impossível) e item sem `valor_total` **e** sem `valor_unit`.
- **[rev. 21/09] `unidade` só aceita `QTD` ou `M2`** (0.8). O OCR vai devolver o que o documento escreve — `UN`, `PÇ`, `m`, `ML`, `m²`. Mapear é trabalho do helper da Fase 6, não do prompt: decisão 11 da Seção 4.

---

4. Decisões já tomadas — não reabrir

As três primeiras são de 16/09/2026. **As dez restantes foram fechadas em 21/09/2026:** seis vinham da antiga Seção 9 — que por isso deixou de ser "decisões em aberto" e virou a lista das conferências que ainda dependem do estado real — e quatro nasceram da conferência contra o DDL.

Da revisão de 16/09

1. **A automação grava pelo sistema, não por REST direto.** Rota de ingestão no gc-sistema, reusando `src/lib/propostas.ts`. O INSERT direto do workflow 4b sai (Fase 6).
2. **Os itens da proposta entram nesta rodada** — extração completa (Fase 5) e gravação em `itens` com `proposta_id` (Fase 6).
3. **A aba "Itens" de `/propostas/[id]` também entra**, mas ela é entrega da **sprint 5, que corre em paralelo.** Combine a fronteira antes de escrever: o desenho abaixo é o que mantém as duas frentes fora do caminho uma da outra.
   - **Desta rodada:** a migration (se alguma faltar), a gravação dos itens pela ingestão, e os helpers puros de item em `src/lib/` (leitura/format/soma), sem React.
   - **Da sprint 5:** a aba, os componentes de tela e o CRUD manual de item.
   - **Ponto de contato:** os helpers de `src/lib/`. Quem chegar primeiro escreve; o segundo usa o que está lá em vez de criar um paralelo. Se a sprint 5 já tiver publicado a aba quando você chegar na Fase 6, o seu trabalho é só fazer o dado aparecer nela.
   - Conferido em 21/09: **não existe `src/lib/itens.ts`**. O terreno está limpo; quem chegar primeiro define o formato.

Da revisão de 21/09 — as seis que estavam em aberto

4. **Onboarding do `chat_id`: cadastro manual pelo admin.** Nada de auto-cadastro com código de vínculo na v1 — isso exigiria estado pendente, expiração de código e uma tela a mais, e dobraria a Fase 7. O admin cadastra o `chat_id` em `/configuracoes/contatos`. Consequência prática: **quem for cadastrado precisa descobrir o próprio `chat_id`** — o bot responde com ele a quem não está cadastrado ainda (Fase 4), e a pessoa passa o número ao admin. Auto-cadastro fica registrado como melhoria, não como pendência desta rodada.
5. **Destino de admin: um grupo do Telegram.** Resolve "quem são os admins" sem redeploy — entra e sai gente do grupo. Custo: **privacy mode desligado no BotFather** (Fase 0, passo 4), senão o bot não lê o grupo. O `chat_id` do grupo é negativo e mora num único nó do `Notificar` (Fase 2). Isso substitui os ~12 hardcodes de `558598202307`.
6. **`PROPOSTA_REGISTRADA`: espelhar o que o fluxo de contrato faz, não inventar valor novo.** Conferido no DDL: o CHECK de `documentos_processamento` aceita só `('PENDENTE','ERRO_VALIDACAO','REVISAO_HUMANA','APROVADO')`, e `PROPOSTA_REGISTRADA` não está lá — o update do 4b **falha hoje.** Na Fase 1, abra o `dOt8aiCX2OCr08RH` e veja o que ele escreve ao registrar um contrato. Se for `APROVADO`, a proposta usa `APROVADO` também, com `proposta_criada_id` preenchido marcando o que foi criado, e **o CHECK não muda.** Só acrescente valor ao CHECK se o 4a provar que precisa de um. Registre o que encontrou no documento de entrega — inclusive se o update vinha falhando em silêncio ou com erro. **Correção de 22/09 (Fase**

0) — a conclusão fica, a premissa não.** O 4a grava `APROVADO`, dentro do CHECK, e portanto **o CHECK não muda**, como a decisão previa. Mas a frase "o update do 4b falha hoje" estava errada: o nó do 4b se **chama** `Supabase - Atualizar (PROPOSTA_REGISTRADA)`, e o corpo que ele envia é `status: "APROVADO"`. `PROPOSTA_REGISTRADA` só existia no rótulo. O update não falhava; o que faltava era preencher `proposta_criada_id` (0 de 29 linhas em gc-dev). Detalhe em `docs/automacao/fase-0-status-entrega.md`, seção 2.3.

7. **Um bot para todas as empresas.** O `empresa_id` sai do `chat_id`, pela `contatos_whatsapp`. Um bot por empresa significaria um webhook e um workflow por empresa, e o schema não pede isso. É o que a Fase 1 já assumia; agora está confirmado.
8. **SLA `TksH6ervKPA7Liar`: auditar sim, reativar não.** A auditoria continua sendo a primeira tarefa da Fase 3 — se ele tiver nós Z-API, eles entram na migração junto com os outros, para não deixar uma bomba armada para quem reativar depois. Mas **reativar o workflow está fora desta rodada** e vai para o documento de entrega como pendência nominal.
9. **O número da proposta é o do documento** (`EB-25-08-0048`), não um gerado pelo sistema. É o número que o cliente cita ao telefone; um `PROP-2026-0NN` paralelo obrigaria todo mundo a traduzir entre dois números para sempre. Consequência: colisão na `unique (empresa_id, numero)` é **resultado esperado**, não erro — a ingestão devolve 409 e o documento vai para `REVISAO_HUMANA` com motivo legível, usando `mensagemDeErroProposta`, que já traduz essa constraint.

Da revisão de 21/09 — as quatro que nasceram da conferência de DDL

10. **`valor_unit` ausente é inferido, não deixado nulo.** `valor_unit = valor_total_lido / quantidade`, com a inferência registrada em `itens.observacao`. Não é preferência: `itens.valor_total` é coluna gerada e deixar o unitário nulo apaga o valor do item (0.8 e 3.5). `quantidade` nula ou zero, ou item sem os dois valores, vai para `REVISAO_HUMANA`.
11. **`unidade` é mapeada no helper, não pedida ao OCR.** `M2`, `m2`, `m²`, `metro quadrado` → `'M2'`; **todo o resto, incluindo ausente,** → `'QTD'`; o valor original do documento vai para `itens.observacao` junto com a nota de inferência, quando divergir do mapeado. O CHECK aceita só `('QTD','M2')` (0.8), e `ML` foi removido de propósito na revisão de schema — não reintroduza.
12. **A automação tem um profile de serviço.** `propostas.created_by` é FK para `profiles`; sem um usuário, a tela mostraria proposta órfã. Criar `automacao@obraminds.com` em gc-dev (Fase 1), perfil `visualizador`, e usar **o uuid dele** em `created_by` **e em `historico.por`**.

    Correção de 21/09: a versão anterior desta decisão dizia "o nome dele em `historico.por`". Está errado — `por` guarda **uuid**, não nome. Conferido em `src/app/(app)/propostas/[id]/actions.ts:168` (`por: auth.userId`), no formato documentado na migration `20260905180000` (`"por": "<uuid do profile>"`) e em `src/components/HistoricoTab.tsx:49`, que resolve o uuid para nome por um mapa de autores. Gravar um nome ali faria a tela exibir "usuário removido".
13. **`n8n-mcp` entra no `.mcp.json` do gc-sistema.** Ver "Como usar". Sem isso as Fases 0, 2, 3, 4, 5 e partes da 6 e da 8 não rodam da sessão que este documento manda abrir.

Da conversa de 22/09 — decididas pelo Breno ao fim da Fase 0

Estas seis (14 a 19) **substituem** o que o plano dizia antes sobre rollback, canal duplo e a lista de 7 workflows. Onde uma fase abaixo contradisser uma delas, vale a decisão.

14. **A Z-API sai por completo, sem período de convivência.** O `Ingestão via WhatsApp (Z-API)` (`pmysfzlAQnKeiLWY`) foi **apagado** em 22/09 — não fica como rollback até a Fase 8. O `Notificar` só tem o ramo Telegram; o campo `canal` segue no contrato de entrada para as chamadas não mudarem de forma, e qualquer valor diferente de `TELEGRAM` é descartado com motivo. Backup JSON dos workflows antigos em `/Users/a1234/n8n-backup/2026-09-22/` (fora do repositório).
15. **Poucos workflows: quanto mais simples, melhor.** Em vez de trocar os 23 nós Z-API um a um nos 6 workflows (a Fase 3 original), o pipeline foi **reescrito em um workflow só**, `Obraminds - Processar Documento`, que substitui Ingestão e Extração, Triagem, Integração Contrato e Integração Proposta. A `Notificação de Erro` virou um Error Trigger dentro do `Notificar`. Alvo: `Notificar` + `Processar Documento` + a ingestão da Fase 4 (+ o SLA, inativo, decisão 8).
16. **Proposta e contrato se ligam fundamentalmente à obra.** Antes, um contrato só era aprovado se citasse uma proposta e o valor batesse; sem proposta citada, ia para revisão. Agora o vínculo que importa é com a **obra** (que vem do contato); a proposta é opcional. Se o contrato citar uma proposta que exista na obra, ela é gravada em `contratos.proposta_origem_id`; **divergência de valor com a proposta não bloqueia** — vai como aviso ao grupo de admin. O banco já estava desenhado assim (`obra_id` NOT NULL nas duas tabelas, `proposta_origem_id` nullable); o que mudou foi a regra do workflow.
17. **A `Resolução Manual de Revisão` sai.** Ela existia para o "contrato ainda sem vínculo com proposta", correlacionava pelo **telefone** e dependia da conversa de vínculo que a Seção 8 já tirou desta rodada. Com a decisão 16, a revisão humana é feita na tela do sistema. Apagada em 22/09. Consequência para a Fase 4: não há mais para onde encaminhar a resposta de texto do cliente.
18. **Nesta rodada o canal recebe só propostas.** Decidido pelo Breno em 22/09: *"não iremos usar os contratos agora, será apenas para subir as propostas no sistema — Telegram recebe → triagem → sobe no sistema e mostra"*. O ramo de contrato (criar contrato, itens e linhas de execução) **saiu** do `Processar Documento`. Documento que o Gemini classificar como contrato vai para `REVISAO_HUMANA` com o motivo "por enquanto recebemos por aqui somente propostas", e nada é criado para ele. Contrato volta numa rodada própria; o desenho do ramo removido está no histórico de versões do workflow e no documento de entrega.
19. **Na Fase 4 o Telegram entra como gatilho do próprio `Processar Documento`**, não como workflow separado. Alvo final da rodada: **2 workflows** — `Notificar` e `Processar Documento` (+ o SLA inativo, decisão 8).
20. **Extração: Gemini gratuito como principal, Groq como reserva.** Decidido pelo Breno em 23/09, depois que a cota gratuita do Gemini esgotou nos testes. O Gemini lê o PDF direto; se falhar (cota, 503, erro), o PDF vira texto (extrator nativo do n8n) e vai para a Groq, modelo `openai/gpt-oss-120b` com `reasoning_effort: low`, **com o mesmo prompt** — o prompt mora num nó só. O documento registra `extrator` e o motivo da falha do Gemini, e o aviso ao grupo diz quando a reserva entrou. PDF sem texto (escaneado) não tem reserva: vai para revisão. Detalhe e limites em `docs/automacao/fase-5-extracao-itens-status-entrega.md`, seção 5.

21. **Até a rota ser alcançável, o n8n grava proposta e itens por REST — com a regra do sistema, não com uma cópia.** Pedido do Breno em 23/09: os itens extraídos têm de ir para as colunas de `itens`. A rota `POST /api/ingestao/proposta` (Fase 6) está pronta, mas o n8n na nuvem não alcança um gc-sistema apontando para o gc-dev. Então o Code node `Montar ingestão` roda **o próprio `src/lib/ingestao.ts`**, empacotado por `scripts/empacotar-ingestao-n8n.mjs` (provado idêntico ao código do repo em 9/9 casos). É uma exceção à decisão 1 na forma (REST), não na regra (a mesma, testada por `node --test`). Quando existir o endereço, o `Montar ingestão` + `Criar proposta` + `Criar itens` viram uma chamada à rota. Na mesma mudança, o envio saiu do sub-workflow `Notificar` para dentro do `Processar Documento`: **1 execução do n8n por documento, não 2.**
22. **Privacy mode do bot de produção religado (revisa a decisão 5).** O Breno religou em 23/09. O privacy desligado só servia para descobrir o `chat_id` do grupo, já anotado; para **enviar** ao grupo não é preciso. Com ele ligado, as mensagens do grupo não chegam mais ao bot e não gastam execução do n8n. O destino de admin continua sendo o grupo.
23. **Contratos pelo bot: reabertos (revoga a decisão 18).** Decidido pelo Breno em 23/09, depois de a sprint 6 entregar o módulo de contratos. Desenho a combinar — ver o documento da Fase 6, seção 5. Até lá, documento classificado como contrato continua indo para revisão.
24. **O PDF recebido fica anexado à proposta criada**, na aba Anexos, como um upload pela tela: bucket `anexos`, path `{empresa}/propostas/{id}/{ts}_{nome}`, entrada no jsonb `anexos` com autor = profile de serviço. Falha no anexo não desfaz a proposta; o aviso ao grupo diz que o PDF não foi anexado.
25. **`DESCARTADO` entra no CHECK de `documentos_processamento`.** A decisão 6 dizia para só acrescentar valor se provasse que precisa; em 25/09 precisou: a revisão pela tela ganhou três saídas (reprocessar, vincular, descartar), e descartar sem apagar mantém o rastro do que chegou. Migration `20260925100000_documentos_revisao.sql`, com `revisado_por` e `revisado_em`.
26. **O n8n grava pelas rotas do gc-sistema (substitui a 21).** Com o endereço público `gc-sistema-nine.vercel.app` apontando para o gc-dev, o `Processar Documento` chama `POST /api/ingestao/proposta` e `POST /api/ingestao/contrato` (Credential `5Kf5wOUxCNenfQkd`, header `x-ingestao-token`). A regra empacotada dentro do n8n e o `scripts/empacotar-ingestao-n8n.mjs` saíram. Volta a valer a decisão 1 na forma, não só na regra.
27. **A obra sai do documento, não do contato (revisa as decisões 7 e 16).** Decidido pelo Breno em 25/09: *"a melhor opção é o bot ler o PDF e fazer a análise, após isso direcionar para o campo correto; o cadastro de um contato é apenas de um funcionário ou admin que terá permissão para enviar os documentos"*. O contato continua dando a **empresa** e autorizando o envio; a **obra** vem do PDF — código, nome ou cliente escritos nele —, casada com as obras da empresa por `identificarObra` (`src/lib/obra-do-documento.ts`), dentro das rotas de ingestão. Só aceita resultado único: nada bateu, ou bateu mais de uma, vira revisão, nunca chute. Obra escolhida na tela (envio ou reprocesso) manda sobre o documento. O formulário de contato perdeu o campo obra; a coluna `contatos_whatsapp.obra_id` fica, sem uso.
28. **Uma obra tem várias propostas e UM contrato vigente** (Breno, 25/09). A rota de contrato do bot recusa (409) o segundo contrato vigente da mesma obra — rescindido não conta — e o documento vai para revisão ("pode ser aditivo ou reenvio"). **Só no bot, por enquanto:** o banco não impede (gc-

dev tem 7 contratos vigentes numa obra, de teste) e a criação pela tela não mudou — pendente de confirmação do Breno.

29. **Conferência do que o bot criou: Aceitar ou Não aceitar** (Breno, 25/09, "leitura A"). O que a automação grava fica "a conferir" (`documentos_processamento.conferencia`, migration `20260925130000`). Aceitar (admin e comercial) mantém; Não aceitar (só admin, é quem pode excluir) apaga a proposta ou contrato, com itens e o PDF anexado, e devolve o documento à revisão com o motivo. Aparece em `/documentos`, no detalhe da proposta e no do contrato. O resultado comercial ("leitura B": aprovada/rejeitada) continua no diálogo de status da proposta, que já existia.
30. **A automação diz em que etapa está e explica o erro em frase de gente** (Breno, 28/09: *"preciso que o log de erros na automação seja descritivo, por exemplo: aconteceu um erro de excesso de execução no n8n, seja mostrado no sistema, e preciso saber em que etapa da automação estamos"*). O documento ganhou `etapa`, `etapa_detalhe` e `etapa_em` (migrations `20260928110000_automacao_eventos` e `20260928120000_automacao_eventos_ordem`). As etapas são: na fila → lendo o PDF (Gemini) → lendo pela reserva (Groq) → registrando no sistema → concluído, aguardando revisão ou parou com erro. A tabela `automacao_eventos` guarda o andamento. O **trigger do banco** grava um evento a cada mudança de etapa, status ou conferência, então quem grava (n8n, rota, tela) não precisa lembrar. O workflow `Notificar` grava, com `origem = 'n8n'`, os erros que nem chegam ao documento. A leitura é por RLS e ninguém insere evento pela API. `descreverErro` (`src/lib/automacao.ts`) traduz o texto cru em título, explicação e o que fazer: limite de execuções do n8n, cota do Gemini, token, obra não identificada, contrato repetido e outros. Aparece no detalhe do documento, numa faixa vermelha em `/documentos` quando um erro que para tudo não teve sucesso depois (`situacaoDaAutomacao`) e em `/documentos/log`.
31. **Obra não identificada vira pergunta pelo bot** (Breno, 07/10, item 2 dos próximos passos). Em vez de ir direto para revisão, o bot manda uma lista numerada e a pessoa responde com o número. A lista tem as obras empatadas, ou até 8 obras da empresa (as mais recentes), e a opção "0) nenhuma destas".
    - **Por que número e não botão:** o nó de Telegram do n8n só monta botões em quantidade fixa, e a lista muda a cada documento.
    - **Onde fica a pergunta:** as opções ficam em `documentos_processamento.opcoes_obra` (migration `20261008100000`), para a resposta ser conferida contra o que foi perguntado.
    - **Resposta:** `POST /api/ingestao/resposta-obra` recebe toda mensagem de texto de um contato cadastrado.
        - Com pergunta aberta naquele chat: o número fixa a obra e devolve o documento à fila, e o n8n relê o PDF.
        - "0": o documento fica em revisão, com o motivo.
        - Resposta inválida: a pergunta é repetida.
        - Sem pergunta aberta: o bot responde o de sempre.
    - **Validade:** a pergunta vale por 7 dias. Depois disso, a revisão pela tela assume.
    - A regra fica em `src/lib/obra-do-documento.ts`, com teste.
32. **Aditivo é uma proposta ligada ao contrato** (Breno, 10/10, opção 1.a).

- A coluna é `propostas.contrato_aditivo_id` (migration `20261012100000`). A FK composta (mesma empresa e obra) fica em `20261012110000`, que **só pode ser aplicada depois do merge**: com duas relações entre `propostas` e `contratos`, junção sem o nome da FK quebra (PGRST201). O código já nomeia `propostas!contratos_proposta_fk`.
- O bot liga sozinho quando o PDF se declara aditivo ou complementar a um contrato da mesma obra (campo `contrato_aditivo` da leitura). Contrato que não existe na obra vira aviso, e a proposta entra como comum.
- Na tela, a proposta mostra "Aditivo do contrato", com "alterar" para admin e comercial, e o contrato lista os aditivos.

33. **Um grupo da equipe, não um grupo por obra** (decidida pelos agentes, 27/10).
    - Grupo liberado não tem obra: a obra continua saindo do PDF (decisão 27), e a pergunta da obra pelo bot (decisão 31) continua valendo quando o PDF não deixa claro.
    - **Motivo:** um grupo por obra obrigaria a manter grupos sincronizados com as obras e criaria uma segunda fonte de obra que pode contradizer o PDF.
    - **Sem custo de mudar depois:** `bot_grupos` aceita vários grupos por empresa.

34. **Liberar por pessoa, não o grupo todo** (27/10). O documento só é aceito se o grupo está liberado **e** a pessoa está liberada.
    - **Motivo:** quem controla os membros do grupo é quem o administra, não o admin do gc-sistema. Liberar o grupo inteiro deixaria qualquer pessoa adicionada mandar PDF que vira proposta ou contrato. A checagem do grupo fica para ninguém usar o bot num grupo qualquer.

35. **Bot fora da administração do grupo, privacy ligado, envio chamando o bot** (27/10). No grupo o PDF só conta com legenda `/enviar` (de preferência `/enviar@<bot>`), menção ao bot ou em resposta a uma mensagem do bot. O resto do grupo é ignorado sem consultar o banco.
    - **Motivo:** o limite de execuções do n8n já parou a automação mais de uma vez (decisão 22). Com o privacy ligado a conversa do grupo não vira execução, e exigir a chamada protege também se o bot virar admin.
    - **A Fase 0 (o que o Telegram entrega em cada um dos três jeitos) é pendência do Breno.** Se só a resposta a mensagem do bot chegar, muda a instrução às pessoas, não o código.

36. **Quem é liberado vale no grupo e na conversa privada** (27/10).
    - **Motivo:** na conversa privada `chat.id` é o id da pessoa, então o `contatos_whatsapp.telegram_chat_id` que já existe é o id da pessoa. Uma permissão, um lugar para revogar, e quem já era cadastrado vale no grupo sem migrar dado.

37. **Técnica: a triagem roda no gc-sistema, não no n8n** (27/10). `triarMensagemTelegram` (pura, testada) roda em `POST /api/ingestao/triagem-telegram`; o nó `Triar mensagem` do n8n vira adaptador de poucas linhas.
    - **Motivo:** padrão da decisão 26: a regra tem uma cópia só, coberta pelos testes e pelo plano de validação, e o n8n já depende do deploy para gravar.

38. **Técnica: o workflow do n8n não é alterado antes do deploy** (27/10). O workflow novo fica num arquivo (`scripts/automacao/n8n-processar-documento-grupo.json`), validado com `validate_workflow`, e o Breno o publica depois do deploy.
    - **Motivo:** `thlKSZcBPX84Y51N` é o de produção. O nó novo chama uma rota que só existe depois do deploy; publicar antes derruba o bot inteiro, inclusive a conversa privada, com 404. Salvar rascunho de workflow ativo também pode republicá-lo.
    - **Ordem:** deploy, variável `TELEGRAM_BOT_USERNAME` na Vercel, publicar o arquivo, teste da Fase 0. O código novo é compatível com o n8n de hoje, então o deploy pode ir antes.
    - **Voltar atrás:** versão ativa antes da mudança `3598b2b4-a6fd-47d0-bdf7-eabb909cb686`.

---

5. Plano por fases

Fase 0 — Bot e credencial

[rev. 21/09] A fase se chamava "Bot, credencial e as decisões de canal". As decisões saíram: são os itens 5 e 7 da Seção 4. Sobrou execução.

0. **Conferir que há ferramentas `n8n_*` na sessão** (decisão 13, "Como usar"). Se não houver, resolva o `.mcp.json` antes de qualquer outra coisa desta fase.
1. Criar **dois bots** no BotFather: um de teste e um de produção. Um token do Telegram admite **um** webhook; testar com o bot de produção derruba o webhook ativo no meio do expediente. Um bot só para todas as empresas (decisão 7) — os dois aqui são teste/produção, não uma divisão por cliente.
2. Cadastrar cada token como **Credential "Telegram API"** no n8n. Nenhum token em nó HTTP.
3. **Criar o grupo de admin no Telegram** (decisão 5), pôr o bot dentro e anotar o `chat_id` — é negativo, e é fácil confundir com erro. Ele substitui o `558598202307` hardcoded em ~12 nós e vai morar num único nó do `Notificar` (Fase 2). Combine com o Breno quem entra no grupo.
4. **Desligar o privacy mode** no BotFather. Não é opcional agora que o destino é grupo: sem isso o bot não enxerga as mensagens dele.
5. Confirmar o comportamento do Telegram Trigger em n8n Cloud: URL de teste e de produção são distintas e ativar/desativar o workflow re-registra o webhook. Saiba disso antes de depurar "o bot não responde".

Aceite: os dois bots respondem `/start`; o bot está no grupo de admin com privacy mode off e o `chat_id` do grupo está anotado; um workflow descartável com Telegram Trigger recebe uma mensagem e mostra o `chat_id` no log.

Fase 1 — Migration de identidade no gc-dev ✅ CONCLUÍDA em 2026-09-21

> **Aplicada em gc-dev**, `migration list` 14/14. Entrega, números das sete camadas e pendências em `docs/automacao/fase-1-contatos-canal-status-entrega.md`. A única coisa desta fase que ficou aberta é a **decisão 6 (`PROPOSTA_REGISTRADA`)**, que depende do n8n e foi

empurrada para a Fase 0/3. O texto abaixo fica como registro do que foi planejado.

Arquivo em `supabase/migrations/`, nome no padrão de timestamp do repo (`2026MMDDHHMMSS_contatos_canal.sql`), tudo dentro de `begin; ... commit;` como as migrations 009/010.

Recomendação: estender, não renomear. Renomear `contatos_whatsapp` quebra os 4 nós n8n que consultam a tabela no exato instante em que a migration roda. Renomear vira item opcional da Fase 8.

```
alter table contatos_whatsapp
  add column canal text not null default 'WHATSAPP'
    check (canal in ('WHATSAPP','TELEGRAM')),
  add column telegram_chat_id text;
alter table contatos_whatsapp alter column telefone drop not null;
-- trocar o unique(telefone) por unicidade por canal:
--   unique parcial em telefone where canal='WHATSAPP'
--   unique parcial em telegram_chat_id where canal='TELEGRAM'
-- + check garantindo que a coluna do canal declarado não é nula
```

Em `documentos_processamento`, adicionar `canal text` e `canal_chat_id text`, e **parar de depender de `dados_extraidos->>telefone`** como chave de correlação (Seção 0.3, item 4). Backfill dos registros existentes: `update documentos_processamento set canal='WHATSAPP', canal_chat_id = dados_extraidos->>'telefone'`.

Acrescente também, na mesma migration, o que a ingestão da Fase 6 vai precisar:

- **`documentos_processamento.proposta_criada_id uuid references propostas(id)`** — o espelho de `contrato_criado_id`, que já existe. Sem isso não há como saber qual proposta veio de qual documento, nem como tornar a ingestão idempotente.
- **`PROPOSTA_REGISTRADA` — decidido (Seção 4, decisão 6):** espelhar o que o `dOt8aiCX2OCr08RH` escreve ao registrar contrato. Abra o workflow, confirme o valor, e se for `APROVADO` **não mexa no CHECK** — a proposta usa `APROVADO` com `proposta_criada_id` preenchido. Registre no documento de entrega o que encontrou, inclusive se o update de hoje vinha falhando com erro ou em silêncio.
- **[rev. 21/09] Corrigir a FK de `contatos_whatsapp.obra_id`.** Hoje é `obra_id uuid references obras(id)` — FK **simples**, ao contrário de todo o resto do schema, que usa composta com `empresa_id`. Do jeito que está, nada impede vincular um contato à obra de outra empresa, e o canal é justamente por onde entra dado não conferido. A Fase 1 já abre essa tabela; é barato fechar o buraco junto: `foreign key (obra_id, empresa_id) references obras(id, empresa_id)`. Confira antes se algum registro existente viola — se violar, o `alter` aborta dentro do `begin/commit` e ninguém se machuca.

[rev. 21/09] Criar o profile de serviço da automação (Seção 4, decisão 12): `automacao@obraminds.com` em gc-dev, com o perfil de menor privilégio que ainda permita leitura. É ele que vai em `propostas.created_by` (FK para `profiles`) e cujo nome vai em `historico.por`. Sem ele, a proposta

criada pela automação aparece sem autor na tela e o histórico registra `por: null`. Guarde o uuid — a Fase 6 precisa dele em `.env.local`.

Nada de coluna nova em `propostas` ou `itens`: a Seção 0.8 confirma que o schema já comporta a proposta e os itens vindos do OCR — **inclusive as três restrições de `itens` que a revisão de 21/09 encontrou**, que são regra de aplicação, não de schema.

Depois: `npm run db:types:dev`. O `migration repair` **não é mais pré-requisito em gc-dev** (0.4) — aproveite a passagem e corrija o texto do `CLAUDE.md` e do `supabase/README.md`, que ainda falam dele como pendência aberta sem dizer que é de prod. A migration correspondente para prod fica **registrada** no documento de entrega como registro para quem operar prod — não como tarefa nossa.

Aceite: migration aplicada em gc-dev, `types.ts` regenerado, `tsc --noEmit` limpo, backfill conferido por query, profile de serviço criado e uuid anotado, e a decisão do `PROPOSTA_REGISTRADA` registrada com o que o 4a faz.

Fase 2 — Sub-workflow `Obraminds - Notificar`

Um workflow novo, `executeWorkflowTrigger` com **`inputSource: passthrough`**.

Contrato de entrada:

```
{ destino: 'CLIENTE' | 'ADMIN',
  canal: 'TELEGRAM' | 'WHATSAPP',
  chat_id, telefone,          // o que estiver disponível
  texto,
  opcoes: null,               // reservado para os botões da Seção 3.3
  contexto: { documentoId, empresaId, origem } }
```

Dentro: um Switch por `canal` → nó Telegram `sendMessage` (usando a Credential) ou o HTTP Z-API existente. O `chat_id` do admin resolve-se **aqui dentro**, num único nó — é o que elimina os ~12 hardcodes.

Preserve `onError: continueRegularOutput` no nó de envio. Todos os 23 nós Z-API têm isso hoje, e é o que impede uma notificação falha de derrubar uma integração já gravada no banco. Perder esse detalhe transforma uma mensagem não entregue em contrato perdido.

Publique o workflow antes de referenciá-lo — o Execute Sub-workflow não enxerga rascunho.

Aceite: `n8n_validate_workflow` com 0 erros; uma chamada manual entrega mensagem no Telegram e outra no Z-API.

> **[22/09] Fase 2 entregue com uma diferença:** sem ramo Z-API (decisão 14). O aceite "uma chamada entrega no Z-API" deixou de existir. Ver `docs/automacao/fase-2-notificar-e-simplificacao-status-entrega.md`.

Fase 3 — Trocar os 23 nós Z-API por chamadas ao `Notificar`

> **[22/09] Substituída pelas decisões 14, 15 e 17.** Os 23 nós não foram trocados um a um: o pipeline virou um workflow só (`Processar Documento`), e os antigos saem. O texto abaixo fica como registro do desenho original. Entrega real em `docs/automacao/fase-2-notificar-e-simplificacao-status-entrega.md` .

Um workflow por vez, validando a cada um. Ordem do mais barato ao mais caro, para que o padrão se prove antes de escalar:

1. Auditar o **SLA** (`TksH6ervKPA7Liar` , desativado) e incluir no plano se tiver Z-API.
2. Notificação de Erro `loR10oopB9vpH22K` — 1 nó.
3. Integração Proposta `I4wdVriPgS2W5MZP` — 4.
4. Integração Contrato `dOt8aiCX2OCr08RH` — 4.
5. Resolução Manual `RikR8XMy9tw8B8O5` — 4.
6. Triagem `MLoVZf7ypPS6Zy3H` — 4.
7. Ingestão e Extração `lsleLGM6t9iKcSLT` — 6.

O texto das mensagens não muda nesta fase. Cada nó Z-API vira um `executeWorkflow` para `Notificar` carregando exatamente o mesmo texto de hoje — assim a comparação antes/depois é 1:1 e qualquer diferença observada é bug, não redação nova. Melhorar as mensagens é uma rodada posterior.

Nós órfãos ficam desconectados, **não deletados**, até a Fase 8.

[rev. 21/09] Aceite, corrigido. A versão anterior pedia "0 referências restantes a `api.z-api.io` fora do `Notificar` ", o que é inalcançável por desenho: a Fase 4 mantém o `pmysfzlAQnKeiLWY` **intacto** como rollback até a Fase 8, com os seus 2 nós Z-API, e o próprio `Notificar` tem um ramo Z-API de propósito (Fase 2). O aceite real é:

- os 6 workflows da lista validando com `n8n_validate_workflow` a 0 erros;
- **0 referências a `api.z-api.io` nesses 6 workflows** — as únicas que sobram no n8n estão no `Notificar` (1 ramo) e no `pmysfzlAQnKeiLWY` (2 nós), e só somem na Fase 8;
- os 23 nós de envio de 0.1 contabilizados um a um: quantos viraram `executeWorkflow` , quantos ficaram órfãos desconectados. Número real no documento de entrega, não "migrados".

Fase 4 — `Obraminds - Ingestão via Telegram`

> **[22/09] Ajustes pelas decisões 14 a 17.** O `pmysfzlAQnKeiLWY` já foi apagado (não há rollback a preservar; o backup JSON serve de referência de topologia). A ingestão **não** encaminha mais para os webhooks `upload-documento` / `resposta-revisao` — chama o `Processar Documento` por Execute Sub-workflow, com `{ empresa_id, obra_id, arquivo_url, canal, chat_id }` . O ramo de resposta de texto perde o destino (decisão 17): responder com orientação ou ignorar é decisão da Fase 4.

> **[22/09, decisão 19] Não é mais um workflow novo:** o Telegram Trigger entra dentro do `Processar Documento` (`thlKSZcBPX84Y51N`), na frente do `Normalizar entrada`, que já espera `{ empresa_id, obra_id, arquivo_url, canal, chat_id }`. Só propostas (decisão 18).

Workflow **novo**. Não edite o `pmysfzlAQnKeiLWY` — ele fica intacto como rollback, do mesmo jeito que o monolito ficou em agosto.

Espelhe a topologia atual nó a nó, com estas substituições:

HOJE (Z-API)	TELEGRAM
Webhook `obraminds/whatsapp-zapi`	Telegram Trigger (`message`, `callback_query`)
`Verificar Tipo de Mensagem` (`body.type` / `fromMe` / `document`)	`message.document` / `message.text`
`Verificar Duplicata` (staticData por `messageId`)	`update_id` — ver ressalva abaixo
`Baixar Arquivo do Z-API` (`documentUrl` + Client-Token)	`getFile(file_id)` → URL de arquivo
`Supabase - Buscar Obra por Telefone` (×2)	busca por `telegram_chat_id`
`caminhoStorage` = `empresa/whatsapp/<messageId>-<file>`	`empresa/telegram/<update_id>-<file>`
`Z-API - Confirmar Recebimento`	`Notificar`

Preserve sem mexer: os dois ramos (documento PDF e resposta de texto), o `Verificar Se Ja Foi Processado` (HEAD no Storage), a heurística de `tipo_documento` por nome de arquivo, e o encaminhamento para os dois webhooks com o payload da Seção 0.2 — acrescentando `canal` e `chat_id`.

Três coisas a tratar explicitamente:

- **Dedup.** `$getWorkflowStaticData('global')` com janela de 200 ids é frágil: zera em restart de worker e não é compartilhado entre execuções paralelas. A rede de segurança real é o `Verificar Se Ja Foi Processado`. Mantenha os dois, não piore, e registre a fragilidade como pendência conhecida.
- **mimeType.** Hoje o default é `'application/pdf'` — um chute. O Telegram manda `document.mime_type`: **rejeite explicitamente** o que não for PDF, com mensagem ao cliente, em vez de assumir.
- **Limite de 20 MB.** IF sobre `document.file_size` antes do `getFile`, com mensagem ao cliente explicando. Comportamento novo (Seção 3.2).
- **[rev. 21/09] `chat_id` desconhecido responde com o próprio `chat_id`.** A decisão 4 da Seção 4 é cadastro manual pelo admin, e isso só funciona se a pessoa souber que número informar. Quando a busca em `contatos_whatsapp` não achar o `telegram_chat_id`, o bot responde algo como *"Seu código de acesso é `123456789`. Passe esse número ao administrador para liberar o envio."* — e **não**

encaminha o documento. Notifique o grupo de admin no mesmo passo, senão o pedido morre no chat da pessoa.

Aceite: um PDF enviado ao bot de teste chega ao `obraminds/upload-documento` com payload idêntico ao do WhatsApp, mais `canal` / `chat_id`.

Fase 5 — Extração completa dos itens da proposta

Só o prompt do Gemini e o `Parsear Resposta da IA`, dentro de `Obraminds - Ingestão e Extração` (`lsleLGM6t9iKcSLT`). Nenhum workflow novo.

Hoje cada item volta com `descricao`, `quantidade`, `valor_unitario`. Passa a voltar com o que o documento tem e o schema aceita (0.8):

```
itens[]: { numero, tipo, linha, acabamento, localizacao, descricao,
           quantidade, unidade, valor_unitario, valor_total,
           largura, altura, vidros, observacao }
```

[rev. 21/09] O que o OCR extrai ≠ o que a ingestão grava. O schema acima é o do *documento*, não o da tabela. Três campos dele **não vão para `itens` como estão**, e confundir os dois é o erro que a revisão de 21/09 encontrou:

CAMPO EXTRAÍDO	O QUE A INGESTÃO FAZ COM ELE
`valor_total`	**Nunca gravado** — `itens.valor_total` é coluna gerada (0.8). Serve de conferência contra `valor_unit × quantidade` e, quando o unitário falta, de origem da inferência (decisão 10)
`valor_unitario`	Vira `itens.valor_unit`. Se nulo, é **inferido** de `valor_total / quantidade` (decisão 10), não deixado nulo
`unidade`	Mapeado para `'QTD'` ou `'M2'` pelo helper (decisão 11). O CHECK não aceita mais nada

Regras de extração, explícitas no prompt:

- **Não inventar.** Campo ausente no documento volta `null`. O modelo preencher `linha: "MERCADO"` por ser o valor mais comum é pior do que vir vazio.
- **`valor_unitario` ausente é comum** (Seção 3.5). Volte `null` — mas **volte sempre o `valor_total`** que o documento traz, porque é dele que a ingestão tira o unitário. Item sem nenhum dos dois é o único caso que não tem conserto e vai para `REVISAO_HUMANA`.
- **[rev. 21/09] `unidade`: extraia o que está escrito, sem normalizar.** `UN`, `PÇ`, `m²`, `ML` — devolva literal. A tradução para `QTD` / `M2` é do helper da Fase 6, que tem teste unitário; o modelo normalizando por conta própria esconde o caso não previsto em vez de expô-lo.
- **`numero`** é o "Item: N" do documento, não a ordem no array — eles divergem quando o documento pula numeração. **[rev. 21/09] `itens.numero` é `integer` e tem `unique (proposta_id, numero)`** (0.8): se o documento trouxer `1.1` / `1A`, ou repetir um número, o insert estoura. Extraia literal e deixe o

helper da Fase 6 decidir — ele converte o que for inteiro, manda para `observacao` o que não for, e devolve `null` no `numero` em vez de inventar uma sequência. Vários nulos convivem no unique.

- Números em formato brasileiro (`R$ 1.986,08`) viram número, não string.

Mexa no prompt com o PDF de referência aberto ao lado: `keen-mendel/relatorios/documentos-fonte/PROP. COM. - EF _ ADITIVOS_15 12 2025.pdf` , proposta `EB-25-08-0048` . **[rev. 21/09 — corrigido] São 16 itens, não 12.** Extraído com `pdftotext -layout` em 2026-09-21: o documento traz `Item: 1` a `Item: 16` , com `Tipo: FC1` a `FC16` . A contagem de 12 estava errada nas três menções deste documento.

O que a extração real mostrou, e que muda a Fase 5:

- **11 dos 16 itens não têm valor unitário; todos os 16 têm valor total.** É a confirmação mais forte possível da decisão 10: gravar `valor_unit` nulo apagaria mais de R$ 115 mil de um documento de ~R$ 121 mil, porque `itens.valor_total` é coluna gerada.
- **A numeração é inteiro simples de 1 a 16** — não há `1.1` nem `1A` neste documento. O caminho de `numero` não-inteiro de `numeroItemDoDocumento` segue coberto só por caso hipotético; se aparecer documento com numeração composta, é aí que se confirma.
- Quantidade vem com zero à esquerda (`01` , `02` , `13`).
- Item 9 é o único com quantidade > 2 e unitário presente (13 × 120,00 = 1.560,00), e serve de controle de que a conta do banco bate com o papel.

A conferência de soma (`quantidade × valor_unitario` vs. `valor_total` , tolerância de 1%) **já existe** na Triagem e continua onde está. Com mais itens tendo `valor_unitario` nulo, o comportamento dessa conferência muda na prática — confira o que a Triagem faz hoje com item sem unitário **antes** de soltar o prompt novo, e ajuste a regra se ela estiver somando zero em silêncio.

> **Correção de 23/09 (Fase 5):** "11 dos 16 itens não têm valor unitário" estava errado. No `pdftotext -layout` , o unitário dos itens 11–16 cai na linha **abaixo** do rótulo `Valor unit.:` , na mesma coluna — a contagem leu o rótulo vazio e perdeu o número. O item 12 prova: unitário R$ 1.986,08 × 2 = total R$ 3.972,16. **Gabarito: 7 itens sem unitário (1–6 e 8), 9 com (7, 9, 10, 11–16).** A decisão 10 continua necessária — 7 itens, ~R$ 106 mil. O prompt da Fase 5 avisa o modelo desse deslocamento. Ver `docs/automacao/fase-5-extracao-itens-status-entrega.md` .

Aceite: o PDF de referência extraído devolve os **16** itens com `tipo` , `linha` , `acabamento` e `localizacao` preenchidos onde o documento traz, e `valor_unitario` nulo exatamente nos itens em que o documento não traz. Confira item a item contra o PDF — é barato e é a única prova real desta fase.

Fase 6 — Rota de ingestão no gc-sistema

O coração da rodada. **Trabalho no gc-sistema**, com as obrigações do `CLAUDE.md` todas valendo.

Route handler, não Server Action — quem chama é máquina, não formulário:

```
POST /api/ingestao/proposta
Header: x-ingestao-token: <segredo>     // .env.local + Credential no n8n
Body:  { documentoId, empresaId, obraId, numero, descricao,
         dataEmissao, dataValidade, valorTotal, desconto,
         condicoesPagamento, pct: { sinal, fd, entregaMaterial, medicaoInstalacao },
         itens: [ ...Fase 5... ],
         origem: { canal, chatId } }
Resp:  201 { ok: true, propostaId }              // criou
       200 { ok: true, propostaId, jaExistia: true }  // idempotente
       401 { ok: false, error }                  // token errado
       409 { ok: false, error }                  // numero duplicado (decisão 9)
       422 { ok: false, error }                  // payload recusado pelas regras
```

Como construir:

- **A regra mora em `src/lib/`, não no route handler.** Um helper puro novo (por exemplo `src/lib/ingestao.ts`) valida o payload e monta as linhas de `propostas` e `itens`, reusando `validarSomaPct`, `validarDesconto`, `pctToFraction` e `novaEntradaHistoricoProposta` de `src/lib/propostas.ts`. Sem `use client`, sem import de React — é o que o torna testável por `node --test`, e esta é a peça que **tem** que ter teste unitário.
- **Autenticação por segredo comparado em tempo constante**, e o cliente Supabase do handler usa a service key — RLS não se aplica a quem não tem sessão. O handler é a fronteira: confira `empresaId` e `obraId` batendo entre si antes de gravar, porque nenhuma policy vai fazer isso por você.
- **[rev. 21/09] Um helper de item também**, e é onde moram as decisões 10 e 11 da Seção 4: inferir `valor_unit` de `valor_total / quantidade` e anotar em `observacao`, mapear `unidade` para `QTD` / `M2`, converter `numero` para integer ou devolver `null`, e **jamais** montar `valor_total` ou `area_m2` na linha do insert (0.8). É a peça com mais casos de borda da rodada; cada regra dessas é um caso de `node --test`.
- **Proposta nasce `rascunho`**, como pela tela (`createProposta` fixa isso). "Chegou pelo Telegram" não é "aprovada".
- **[rev. 21/09] Histórico desde o nascimento — e a tela não faz isso hoje.** Conferido: `createProposta` (`src/app/(app)/propostas/nova/actions.ts:38-46`) grava `created_by` e `status`, e **não escreve entrada nenhuma** em `historico` — o histórico só ganha linha na primeira transição. Então a ingestão está inaugurando um padrão, não copiando um:
  - `novaEntradaHistoricoProposta` exige `de` e `para`, ambos `PropostaStatus`. Para uma criação, use `de: 'rascunho', para: 'rascunho'` — a entrada marca a origem, não uma transição.
  - `por` recebe o **uuid** do profile de serviço (decisão 12, corrigida em 21/09 — `por` guarda uuid, e a tela resolve o nome). **[23/09] Corrigido aqui:** esta linha dizia "o nome".
  - O `documentoId` entra no rastro. `EntradaHistorico` não tem campo para ele, então **decida e registre**: ou o `documentoId` vai embutido em `por` (`automação (Telegram) · doc <id>`), ou o vínculo fica só em `documentos_processamento.proposta_criada_id`, que já é uma seta navegável nos dois sentidos. A segunda é mais limpa e não distorce o tipo; prefira-a salvo motivo.

    - `created_by` recebe o uuid do profile de serviço. Nulo passaria no schema, mas deixa a proposta sem autor na tela.

- **Idempotência por `documentoId`**: se `documentos_processamento.proposta_criada_id` já estiver preenchido, responda 200 com o id existente em vez de criar a segunda proposta. O n8n reexecuta mais do que se imagina.
- **Proposta e itens na mesma unidade lógica.** Se a gravação dos itens falhar depois da proposta criada, ou desfaz ou marca o documento em `REVISAO_HUMANA` com motivo — proposta meio gravada é o pior resultado possível. Prefira uma função SQL transacional se for o caminho mais curto para isso; decida e registre.
- **`as`, nunca `as unknown as`** no retorno de `select()`.

No n8n, o `I4wdVriPgS2W5MZP` perde os nós de INSERT direto em `propostas` e ganha um HTTP Request para a rota. Os nós antigos ficam **desconectados, não deletados**, até a Fase 8.

Obrigações de validação desta fase — todas, não só as convenientes:

- **Camada build:** `/api/ingestao/proposta` é rota nova — rode `bash scripts/validar.sh build --aceitar-rotas` para regravar `scripts/rotas-esperadas.txt` (a baseline sai de **31** para 32; conferido, hoje o arquivo tem 31 linhas e já inclui as cinco rotas `/api/*` existentes).
- **[rev. 21/09] Camada runtime: a rota NÃO passa em `scripts/validacao-rotas.json` como está.** A versão anterior mandava acrescentá-la ali sem ressalva, e isso reprova. O `scripts/validar-runtime.mjs:250` faz, para **toda** rota do arquivo, uma chamada **sem sessão** e exige `307` / `302` com `location` contendo `/login`. As `/api/export/*` passam porque são autenticadas por sessão de cookie; `/api/ingestao/proposta` é autenticada por `x-ingestao-token` e devolve **401** — e um GET numa rota só-POST devolve **405**, não os `esperaStatus: 200` do default. Duas saídas, escolha e registre:

  1. **Acrescentar um opt-out ao script** — um `"esperaAnon": 401` (ou `false`) lido pelo bloco do anon check, e `"esperaStatus": 405` na entrada. Cobre a rota de verdade e serve para a rota de contrato da Seção 8. Prefira esta.
  2. **Deixar a rota fora de `validacao-rotas.json`**, coberta só pelas camadas build e escrita — e então **listar nominalmente** no documento de entrega que `/api/ingestao/proposta` não tem cobertura de runtime, que é o que a regra 3 do `CLAUDE.md` exige.

  O que **não** vale é acrescentar a entrada e deixar a camada vermelha, nem inventar um gate de ambiente que pule a checagem: é exatamente o skip silencioso que a `auditoria-cobertura-sprint-4.md` documenta (Seção 6).
- Passos novos em `scripts/validar-escrita.mjs`: ingestão feliz criando proposta **e** itens, payload inválido sendo recusado (soma de pct > 100%, número duplicado, obra de outra empresa), token errado devolvendo 401, e a segunda chamada com o mesmo `documentoId` devolvendo o mesmo id. **[rev. 21/09] Mais três, das decisões novas:** item só com `valor_total` gravando `valor_unit` inferido e `observacao` marcada; item com `unidade: 'UN'` gravando `'QTD'`; item com `quantidade: 0` e sem unitário mandando o documento para `REVISAO_HUMANA` em vez de gravar. **Com limpeza do que criar** — a proposta e os itens de teste saem no fim, como os outros passos já fazem.

- Query nova em `scripts/validar-dados.mjs` se a rota ler algo que nenhuma page já lê.
- Teste unitário dos dois helpers puros em `src/lib/` — o de proposta e o de item. O de item cobre, no mínimo: inferência de `valor_unit`, os dois casos que vão para `REVISAO_HUMANA`, o mapa de `unidade` (incluindo `ML` → `QTD` e ausente → `QTD`), `numero` não-inteiro virando `null` com registro em `observacao`, e a **ausência** de `valor_total` / `area_m2` na linha montada.

Aceite: `npm run validar` inteiro, com os números novos registrados no documento de status do bloco, comparados com a baseline de 0.7. Uma proposta criada pela rota abre em `/propostas/[id]` com valor, obra, histórico e — se a aba da sprint 5 já existir — os itens.

> **[23/09] Duas dependências que esta seção não previa.**
>
> 1. **Sessão paralela.** A sprint 5 está ativa no mesmo working tree. Uma rota nova faz a camada build dela reprovar (diff de rotas), e um teste novo muda a contagem dela. A Fase 6 foi escrita e testada **fora do repo**, em `/Users/a1234/gc-sistema-staging/fase-6/`, e entra quando a outra sessão parar.
> 2. **Endereço público.** O n8n roda na nuvem; para chamar a rota, o gc-sistema precisa estar publicado apontando para o **gc-dev**. O único deploy hoje é o da Vercel, que aponta para gc-prod (fora do escopo) e tem login da Vercel na frente. A rota se testa inteira por HTTP local (camadas escrita/runtime); **trocar o REST direto do n8n pela rota espera essa decisão.** Ver `docs/automacao/fase-6-rota-ingestao-status-entrega.md`.

Fase 7 — Painel no gc-sistema

Duas telas, seguindo o padrão de `/orcamentos`, `/fd` e `/propostas`: estado todo na URL, busca com debounce de 300 ms, `PAGE_SIZE = 20`, `Pagination` compartilhado, período por `computePeriodoCutoff` de `@/lib/listagem`.

- **`/documentos`** — listagem de `documentos_processamento`: status, tipo, canal, obra, valor, data. Filtro por status. Detalhe com `dados_extraidos` renderizado, link assinado do arquivo e **link para a proposta criada** (`proposta_criada_id`, Fase 1) — é o que fecha o ciclo visualmente: documento recebido → proposta cadastrada.
- **`/configuracoes/contatos`** — CRUD dos contatos autorizados. Conferido no DDL (`20260901120910_contatos_whatsapp.sql`): as policies já existem e são **select para a empresa toda, insert/update para admin e comercial, delete só admin**. A tela segue essas policies, e o guard de layout protege a rota enquanto **cada Server Action repete a checagem de perfil por conta própria** (`CLAUDE.md`).
- **[rev. 21/09] Onboarding do `chat_id`: cadastro manual** (Seção 4, decisão 4). Não há contato pendente nem código de vínculo — o formulário ganha os campos `canal` e `telegram_chat_id` (Fase 1), e o admin digita o número que a pessoa recebeu do bot (Fase 4). O formulário valida que **o campo do canal declarado está preenchido**, que é a mesma regra do CHECK da migration; deixar o banco recusar sozinho daria erro cru na tela. Auto-cadastro com código fica registrado como melhoria no documento de entrega.

Obrigações que vêm com o `CLAUDE.md` do gc-sistema, todas:

- Rota nova em `scripts/validacao-rotas.json` (rota + trechos de HTML que provam o render) e `bash scripts/validar.sh build --aceitar-rotas` para regravar `scripts/rotas-esperadas.txt`.
- Query nova em `scripts/validar-dados.mjs`, **copiada do `page.tsx`**.
- Server Action nova em `scripts/validar-escrita.mjs`, com limpeza do que criar.
- Tela ou diálogo novo em `scripts/validar-navegador.mjs`.
- Regra de permissão por perfil como rota com `"perfil": "<nome>"`.
- Helpers puros em `src/lib/`, sem `use client` e sem import de React.
- Guard de layout protege a rota; **a Server Action repete a checagem de perfil por conta própria**.
- `as`, nunca `as unknown as`, no retorno de `select()`.

Aceite: `npm run validar` inteiro passando, com os números reais por camada registrados no documento de status do bloco e o que não rodou listado nominalmente.

Fase 8 — Corte

1. Teste end-to-end real pelo Telegram, os **três** caminhos: proposta aprovada automaticamente **chegando cadastrada com itens em `/propostas`**, contrato aprovado automaticamente, e `REVISAO_HUMANA` seguido de resposta. Use PDFs de `keen-mendel/relatorios/documentos-fonte/` — **um documento que ainda não foi processado**, senão bate na unique de `contratos.numero` (ou na `propostas_empresa_id_numero_key`) e o teste falha por motivo errado. Confira o resultado por `n8n_executions` e por query no banco, não pela mensagem recebida.
2. Só depois: desativar `Obraminds - Ingestão via WhatsApp (Z-API)`, remover o ramo `WHATSAPP` do Switch do `Notificar`, apagar os nós órfãos das Fases 3 e 6.
3. Não delete o workflow Z-API. Deixe desativado como rollback por pelo menos um ciclo.
4. Opcional, depois de tudo estável: renomear `contatos_whatsapp` → `contatos_canal`.

Estado em 12/10:

- **Item 1, os três caminhos: feito.**
  - Proposta PROP-2026-0117 e contrato CT-2026-0042 (Vista Verde, 07/10) chegaram com itens e PDF em Anexos.
  - O caminho de revisão, com a pergunta da obra e a resposta "0", foi feito com o CT-2026-0051 em 10/10.
- **Itens 2 e 3: feitos antes.** O workflow Z-API saiu em 22/09, a pedido do Breno, e o `Notificar` ficou só com o alerta de erro.
- **Workflow `SLA Gatilhos de Execução (5 dias)`**: estava inativo e foi **apagado** em 12/10. A cópia está em `~/gc-sistema-staging/n8n-backups/`. O alerta de prazo de execução, se voltar, é assunto da trilha de sprint (`/execucao`).
- **Bot de teste:** a Credential "Telegram — ObraMinds Ingestão (teste)" saiu do n8n e o bot foi apagado no BotFather pelo Breno em 12/10.
- **31 documentos de teste parados:** descartados em 12/10, com autorização do Breno.
- **Corte da Fase 8 concluído.**

- **Workflows em uso:** `Processar Documento` e `Notificar`.

---

6. Armadilhas conhecidas (custaram tempo antes)

Do n8n:

- **Execute Sub-workflow**: publique o sub-workflow **antes** de referenciá-lo, e use `inputSource: passthrough`.
- **n8n divide arrays automaticamente**: uma resposta HTTP que é array JSON vira *um item por elemento*. `$input.item.json` devolve só a primeira linha. Use `$input.all().map(i => i.json)`. Esse bug exato derrubou a primeira aprovação automática que chegou ao fim do fluxo — e o array de itens da Fase 5 é exatamente a forma de dado que o dispara.
- **`n8n_health_check` mente**: retorna `connected: true` com API key inválida. Só reiniciar o Claude Code recarrega a env.
- **`numero_contrato` no payload é o número do próprio documento** (vale para proposta e contrato), não o número de um contrato referenciado.
- **Não reintroduza `order=created_at.desc&limit=1`** no `Supabase - Buscar Proposta da Obra` da Triagem. O filtro `numero=eq.` é incondicional de propósito (sentinela `__SEM_PROPOSTA_REFERENCIADA__`); o fallback antigo cruzava documentos sem relação.
- **`Buscar Documento Pendente` tem um `limit=1`** que já é conhecido como frágil quando há mais de um documento pendente para o mesmo contato. Não conserte de passagem — está previsto no desenho conversacional (migration 012).
- **Notificação falha não pode derrubar o fluxo**: `onError: continueRegularOutput`.

Do gc-sistema (aprendidas na sprint 4 — `auditoria-cobertura-sprint-4.md`):

- **Gate de ambiente em script de validação vira skip silencioso.** Três checagens de permissão passaram dois dias imprimindo "PULOU" e devolvendo exit 0. Checagem que depende de variável ausente tem de **estourar**, não pular.
- **O toast intercepta o clique** na camada de navegador: ele fica sobre a faixa de botões do header e o clique é por coordenada. Espere o toast sair antes do próximo passo.
- **Linha de tabela não é link.** O `DataTable` navega por `onClick` com `router.push`; o seletor certo é `table tbody tr`, não `a[href^="..."]`.
- **`npm test` não confere tipo** — `node --test` remove os tipos sem validar. Um teste verde com campo inexistente só cai no `tsc`, que roda antes por desenho.
- **`valor_final` é coluna gerada e os `pct_` são frações 0–1** (0.8). Os dois erros passam pelo TypeScript e só aparecem no banco.
- **[rev. 21/09] Em `itens`, `valor_total` e `area_m2` TAMBÉM são geradas** — e essa é a que custou uma versão inteira deste documento. A versão anterior listava as duas como campos graváveis e mandava, para item sem unitário, "deixar o `valor_total`". Resultado seria item com valor nulo no

banco e ninguém percebendo, porque a tela mostraria `R$ 0,00` sem reclamar. Regra: **na linha do insert de `itens` não existe `valor_total` nem `area_m2`.**

- **[rev. 21/09] `itens.unidade` aceita só `QTD` e `M2`**, e `numero` é `integer` com unique por proposta (0.8). Os três são CHECK/índice: passam pelo `tsc`, passam pelo `npm test`, e só estouram no `insert` — ou seja, na camada escrita, que é justamente a que roda por último entre as que conectam.

7. O que não fazer

- Tocar em gc-prod, sob qualquer justificativa.
- Commitar ou dar push sem pedido explícito.
- Deletar o workflow Z-API, nós órfãos ou colunas antes do teste end-to-end passar.
- Editar `pmysfzlAQnKeiLWY` em vez de criar um workflow novo.
- Reescrever o texto das mensagens na mesma rodada em que troca o canal.
- Reescrever regra de proposta em JavaScript de nó n8n: ela mora em `src/lib/propostas.ts` e a ingestão a reusa (Seção 3.4).
- Construir a aba "Itens" de `/propostas/[id]` por conta própria — é da sprint 5, que corre em paralelo (Seção 4, decisão 3).
- Implementar os botões do fluxo conversacional agora (Seção 8).
- Tratar `keen-mendel/relatorios/v0_zapi_whatsapp/obraminds_ingestao_whatsapp_zapi.json` como fonte de verdade — está desatualizado.
- **[rev. 21/09] Gravar `valor_total` ou `area_m2` em `itens`** — são colunas geradas (0.8).
- **[rev. 21/09] Reintroduzir `ML` em `itens.unidade`** — foi removido de propósito na `revisao_schema`, com migração dos dados existentes para `QTD`.
- **[rev. 21/09] Reabrir as decisões da Seção 4.** As treze estão fechadas. Se alguma parecer errada na hora de implementar, **pare e avise** — não decida diferente em silêncio, porque várias delas amarram fases distintas uma na outra.

8. Fora de escopo, mas não esquecido

O fluxo conversacional de vínculo contrato↔propostas com inline keyboard fica para a rodada seguinte. Depende das migrations `011` (`propostas.contrato_id`) e `012` (`pergunta_pendente` / `opcoes_apresentadas`), ambas ainda não criadas. Spec em `keen-mendel/relatorios/md/fluxo_vinculo_conversacional.md` e `keen-mendel/relatorios/md/modelo_vinculo_contrato_propostas.md`.

O que esta rodada deve garantir: o campo `opcoes` no contrato do `Notificar` (Fase 2) e o `callback_query` já registrado no Telegram Trigger (Fase 4). Custa nada agora e evita reabrir os 6 workflows depois.

Também fora: **ingestão de contrato pela rota do sistema.** O 4a continua gravando direto por REST nesta rodada — a proposta é o caminho que se está fechando. Quando a rota de proposta estiver estável, a de contrato é o mesmo desenho com `contratos`, `itens` e `execucao`, e aí o INSERT direto sai do sistema inteiro.

9. O que ainda depende de confirmação durante a execução

As seis decisões que esta seção listava como abertas foram fechadas em 21/09/2026 e viraram os itens 4 a 9 da Seção 4. Não há mais decisão de escopo pendente. O que sobrou são três conferências que só o estado real responde — nenhuma delas é escolha, todas são "olhe antes de escrever":

1. **O que o `dOt8aiCX2OCr08RH` escreve ao registrar um contrato** (Fase 1). A decisão 6 já diz o que fazer com cada resposta possível; só falta abrir o workflow. Se o 4a usar um valor que também não está no CHECK, aí sim **pare e avise** — significa que os dois fluxos estão quebrados e o conserto é maior que esta fase.
2. **Se o SLA `TksH6ervKPA7Liar` tem nós Z-API** (Fase 3, primeira tarefa). Se tiver, eles entram na migração; o workflow continua desativado de qualquer jeito (decisão 8).
3. **O que a Triagem faz hoje com item sem `valor_unitario`** (Fase 5). A conferência de soma de 1% existe lá e passa a receber mais itens sem unitário. Se ela estiver somando zero em silêncio, ajuste a regra **antes** de soltar o prompt novo — senão a Fase 5 entrega itens corretos que a Fase 4 do fluxo rejeita.

E duas escolhas de implementação que a Seção 5 pede para **decidir e registrar**, não para perguntar: a forma de transação entre proposta e itens (Fase 6) e a saída da camada runtime para `/api/ingestao/proposta` (Fase 6, opções 1 e 2).

10. Referências

Os oito arquivos do keen-mendel foram conferidos em 21/09/2026 e existem. O caminho completo é `/Users/a1234/Documents/antigravity/keen-mendel/` + o que está abaixo.

[rev. 21/09] Fonte de verdade do schema, e a única que vale para `itens`:

- `supabase/migrations/20260424121550_initial.sql` — DDL original de `propostas` e `itens`.
- `supabase/migrations/20260511151924_revisao_schema.sql` — **leia a §6 e a §8 antes de escrever qualquer insert.** É ela que torna `itens.valor_total` e `itens.area_m2` colunas geradas, restringe `unidade` a `('QTD','M2')`, cria o unique de `(proposta_id, numero)` e acrescenta `itens.observacao`. Nenhuma dessas mudanças aparece no DDL original, e foi exatamente por ler só o original que a versão anterior deste documento errou a Fase 5.
- `supabase/migrations/20260901120910_contatos_whatsapp.sql` — tabela e policies do canal.
- `scripts/validar-runtime.mjs` — o anon check da linha ~250, que decide o desenho da cobertura de runtime da Fase 6.

Do keen-mendel:

- `keen-mendel/relatorios/v0.1/fluxo_workflows_v0.1.md` — mapa dos 7 workflows pós-split.
- `keen-mendel/relatorios/md/fluxo_e_campos_extracao.md` — o que a IA extrai hoje, campo a campo.
- `keen-mendel/relatorios/md/roadmap_split_workflow_ocr.md` — o split de agosto, cujo procedimento de corte (validar → testar → só então apagar) este plano repete.
- `keen-mendel/relatorios/md/fluxo_vinculo_conversacional.md` — §3 e §4.
- `keen-mendel/relatorios/md/modelo_vinculo_contrato_propostas.md` — DDL das migrations 011/012.
- `keen-mendel/relatorios/md/sessao_26082026_checkup_e_vinculo.md` — checkup dos workflows.
- `keen-mendel/relatorios/documentos-fonte/PROP. COM. - EF _ ADITIVOS_15 12 2025.pdf` — proposta real de **16** itens (a contagem de 12 que este documento trazia estava errada; conferida por `pdftotext -layout` em 2026-09-21), referência da Fase 5.
- `gc-sistema/CLAUDE.md` e `gc-sistema/docs/tecnicos/plano-validacao.md`.
- `gc-sistema/docs/sprint-4/sprint-4-apresentacao-gestao.md` — o que a sprint 4 entregou.
- `gc-sistema/docs/tecnicos/auditoria-cobertura-sprint-4.md` — os modos de falhar do próprio plano de validação.